Drawing Jazz Chords

Justin Bruss

November 2023

1 Abstract

The goal of this project is to expose the underlying structure of musical chord transitions in jazz using data from a large number of jazz pieces and, in the process, to explore and implement various techniques for drawing large graphs (both for fun and for gaining knowledge/experience). Traditional music theory is derived from Western music from roughly the period 1650 to 1900 and is well described/structured [1]. Modern music styles (including jazz) are heavily influenced by classical Western music with the use of chords and tonal centers [1]. Although jazz is much less structured than classical music, one can imagine that, given enough data, the underlying principles of tonal harmony from classical music still show up in jazz—this is the core idea that inspired this project. The source of the jazz chord data can be found at [2]. The code for the project is written in the Rust Programming Language and can be found at [3].

2 Introduction

2.1 Music Theory

As stated earlier, traditional/classical music theory is based on Western music produced from roughly 1650 to 1900 [1]. "It is the study of what music is, and how musicians and composers make music" [4]. It tells why certain notes sound "good" (or consonant) together while others sound "bad" (or dissonant) together. At the very core of classical music theory is the circle of fifths which places the twelve chromatic pitches on a circle such that adjacent pitches are a perfect fifth (seven chromatic notes) apart (see Figure 1). While there are many patterns and symmetries that show up in this representation, we mostly care about what it means in terms of the sequences of chords that it dictates. In classical music theory, a tonal sequence is a sequence of chords that are within a single key [1]. Chord progressions that maintain a tonal center are tonal sequences (that is, if a section of a piece is in a certain key (and only in that key), it must be the case that the chord progression of the section is composed of chords that are within that key). For the two most used scales, major and minor, there are 7 different notes which can be found for a specific key based on

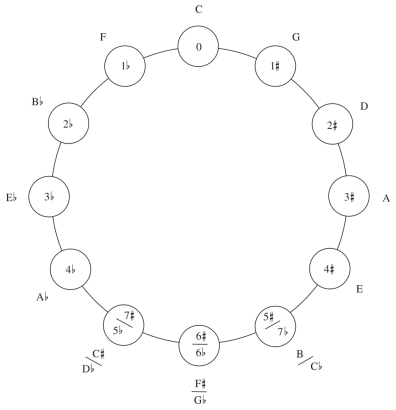

Figure 1: The circle of fifths [1]

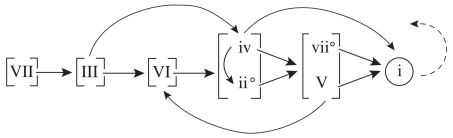

Figure 2: A flow diagram of the rules for creating tonal sequences for minor scales (note that **i** can jump to any other chord including itself) [1]

its position in the circle of fifths. These seven notes can be used to create seven unique base chords (represented as roman numerals) for each key. There are also rules which dictate how the seven chords in a key can transition from one to another based on their sonic characteristics (an example is shown in Figure 2 for minor keys). This approach to chord progressions gives us a structure that shows us the role of each chord in a scale. For example, the **V** chord almost always transitions to a **I** or **i** chord for major and minor scales respectively because there is a tension in the **V** chord that is *resolved* when moving to the **I** chord which makes it sound natural [1]. This transition can often be found at the end of a sequence (and because it's so common, has the name authentic cadence). An even stronger tension can be created by using an alternate form of the **V** chord called the $\mathbf{V}^7$ or dominant seventh chord which adds an additional note a 7th above the base note of the **V** chord. Dominant seventh chords almost always resolve to the **I** chord because this movement releases the tension and sounds pleasing to the ears. One can say that the **V** and $\mathbf{V}^7$ chords function very similarly because they can both be substituted to create the feeling of resolution at the end of a sequence of chords.

2.2 Project Idea

As we've shown in the previous subsection, classical music theory is highly structured in a way that determines when and where certain notes/chords can be used based on the sonic characteristics of the harmonies they create. As stated previously, many modern styles of music are influenced by classical music theory—including jazz. Unlike classical music where pieces are organized, notated, and arranged in a way such that a performance is an interpretation of an existing composition, jazz is performed as an improvisation. There aren't any clearly defined rules to jazz music, however, the style still contains elements of harmony from classical music [5]. This implies that there may still be patterns

that emerge in jazz chord progressions that come from the sonic characteristics of the chords themselves. It's possible that we may be able to extract some of that structure by plotting the chord progressions of many jazz pieces as a graph. This project is an exploration of data as well as a learning experience utilizing multiple graph drawing techniques and algorithms to hopefully achieve an interesting result.

3 Implementation

Each of the following sections progress in a sequential order where each implementation is used to produce an improved[1] graph using the subsequent implementation. All of the code is written in The Rust Programming Language at can be found at [3].

3.1 Data Processing

As stated earlier, data for the jazz pieces is taken from the JazzStandards github repository [2]. This data set contains 1382 JSON formatted jazz pieces formatted as sections in terms of their chord progressions. For example, the piece "Dedicated to You" by Chaplin, Cahn and Zaret is formatted as:

```
{
    "Title": "Dedicated To You",
    "Composer": "Cahn-Chaplin-Zaret",
    "Key": "Bb",
    "Rhythm": "Ballad",
    "TimeSignature": "4/4",
    "Sections": [
      {
        "Label": "B",
        "MainSegment": {
          "Chords": "Bbmaj7|Eb7(Em7),(A7)|Bbmaj7|Dm7,G7|Cm7b5(Ebm7),
                     F7b9(Ab7)|Bbmaj7,,Am7,D7"
        },
        "Endings": [
          {
            "Chords": "Gm7,C7|Cm7,,F7#5(F#m7),(B7)"
          },
          {
            "Chords": "Gm7,C7,Cm7,F7|Bb6,,Em7,A7"
          }
        ]
      },
      ...
```

[1] In the sense that it is leading to a graph that is potentially more human readable

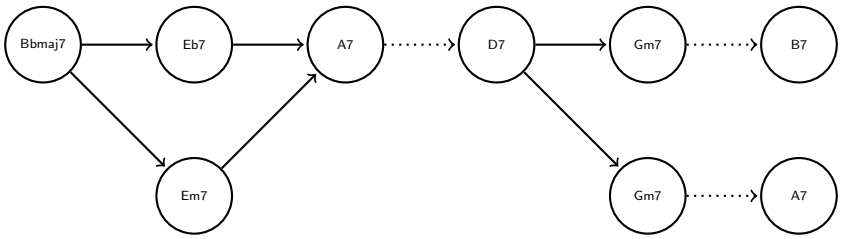

Figure 3: Flow diagram of the "B" section of "Dedicated to You". Note that the () notation indicates that a chord can be substituted for another chord which is why **Eb7** and **Em7** are placed the way they are.

```
    ]
}
```

From here, we want to build a graph where unique chords are nodes and transitions from one chord to another chord are directional edges. To do this, we can represent each section of a piece as a flow graph of chords. For example, the "B" section of "Dedicated to You" shown above can be represented as shown in Figure 3. From the 1382 pieces, we get 3960 different flow graphs (which implies that there are 3960 sections) and 1010 unique chords. Now, we can build the main graph representing the data from all pieces by treating each unique chord as its own node. This is done by keeping track of chords and their associated edges in a set and traversing the nodes of each flow graph—if a node is a chord we haven't encountered yet, we add it to our set of chords, if it has been encountered, we add any outgoing edges to the existing chord in the chord set.

Now that we have created our graph, we can start working on drawing it. As a quick sanity test, we can plot the data as a circular graph. This is shown in Figure 4. Note that it is quite hard to make out what is happening because there are over 50k edges in the graph. The goal of the following implementations is to try and organize and make sense of what is happening here.

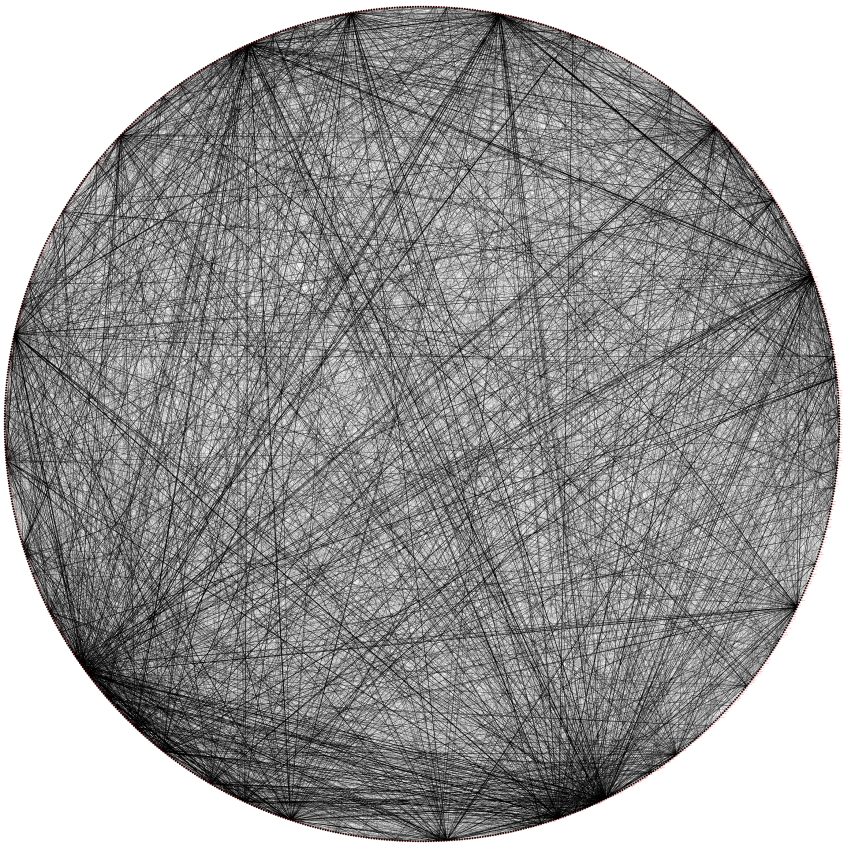

Figure 4: The chord data drawn as a circular graph. Note that edge thickness is used to represent multiple edges going between nodes and is logarithmically scaled.

3.2 SVD

We want to somehow draw this graph in a way that removes the clutter from the image to make it more human readable. As we've shown in the section about music theory, chords have innate sonic properties that determine how they fit within chord progressions. The sonic properties of a chord C are often reflected in how often certain chords transition to C. We can use this to determine how similar one chord is from another. For example, if chord A often transitions to chord B and A also often transitions to chord C, it is likely that B and C have similar sonic properties that make both transitions sound "good".

To make use of this property, we can represent our graph as a directional adjacency matrix M_a where $M_a[i, j]$ is the number of edges pointing to node i

from node j. In this representation, we can think of each row r of the matrix as a vector representing how all other chords transition to r. Using the idea from the previous paragraph, it should be the case that row vectors in M_a that point in a similar direction have similar sonic properties since the distribution of how chords transition to them is similar. We then normalize all of the rows to have unit magnitude to get M. This places similar chords close together in the 1010 dimensional chord space since each chord is now represented by a direction and chords in M are similar if they point in similar directions.

At this point, the matrix M contains a representation for all 1010 unique chords such that similar chords should be physically close together in euclidean space. However, as it is quite difficult for humans to visualize a 1010 dimensional space, we must somehow reduce the size of the space to something reasonable like 2D. There are a variety of methods for multi-dimensional scaling but one of the most straight forward is singular value decomposition. Given M, we can represent it as:

$$M = USV^T$$

Where S is a $n \times m$ diagonal matrix of singular values sorted from largest to smallest and m is the number of singular values required to represent M, U is a $n \times n$ matrix where the columns are the left singular vectors corresponding to the singular values, and V is a $m \times m$ matrix where the rows are the right singular vectors corresponding to the singular values. In addition, U and V are orthogonal bases. In theory, we can reduce the dimensionality of a space while retaining some of the behavior of that space by preserving the most significant singular values and their associated vectors. If we want to reduce an n dimensional space spanned by matrix M to a k dimensional space spanned by matrix M_k ($k < m$), we can select the k largest singular values and project the points in the space onto a k dimensional space spanned by k vectors in V corresponding to the k singular values. To do this, we have the following equation:

$$M_p = U_k S_k V_k^T$$

where U_k is the first k columns of U, S_k is the upper $k \times k$ matrix in S and V_k^T is the first k rows of V^T. The result, M_p is a $n \times m$ matrix where each row r_p of M_p is the original row r of M projected into the space spanned by the row vectors of V_k^T. Using this idea, we can extract a k dimensional representation of the data by replacing V_k^T with I_k (the $k \times k$ identity matrix) to get a $n \times k$ matrix M_k where each row r_k of M_k is the coordinate representation of the corresponding row r in M in the space spanned by the rows of V_k^T. This gives us:

$$M_k^* = U_k * S_k \tag{1}$$

to get a matrix where each row in M_k^* is a k dimensional representation of the corresponding row in M such that M_k^* preserves the behavior of the k strongest singular values in M.

From here, we can use $k = 2$ to plot the points of our original normalized adjacency matrix onto a 2D grid. The result is shown in Figure 7.

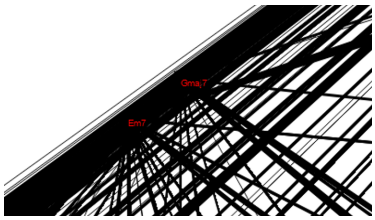

Figure 5: Em7 and GM7 in SVD drawing

Figure 6: Dm7 and FM7 in SVD drawing

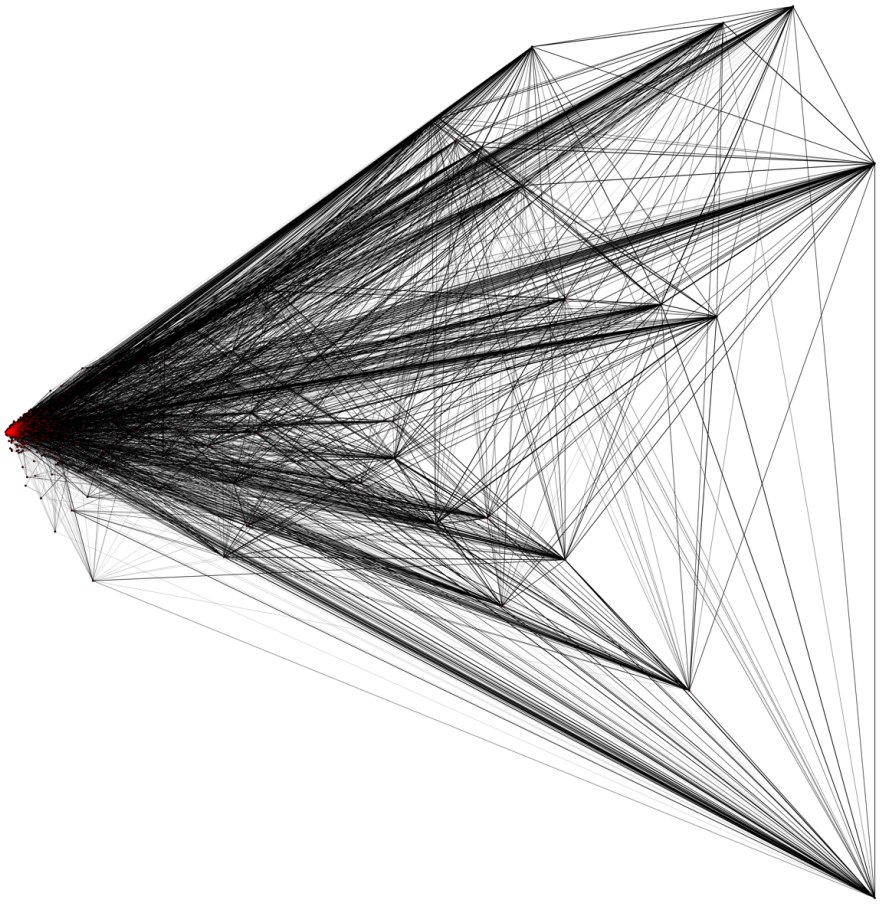

Figure 7: The chord data drawn using SVD to reduce the dimensionality of the row-normalized adjacency matrix. This drawing leaves a lot to be desired.

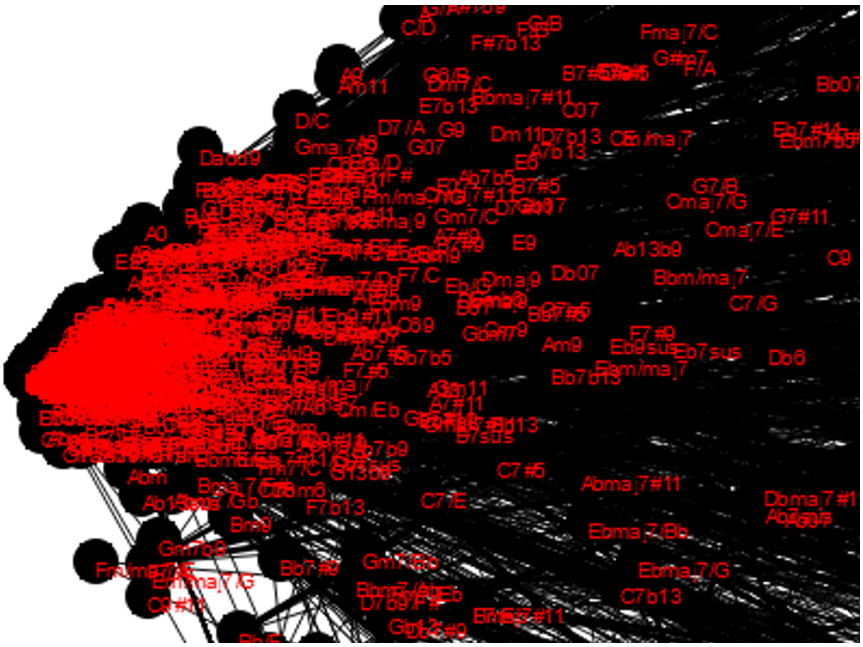

Figure 8: Most of the chords ended up near the origin

This is not a very satisfying result. It seems that most of the behavior of chord interactions could not be captured using just two latent factors as most of the points representing chords ended up bunched together near the origin as shown in Figure 8. However, there are some promising signs as some clear behavior does seem to have been captured which is shown in Figure 5 and Figure 6. Em7 and GM7 are very similar chords as they share 3 notes in common (both have 4 notes) and a similar argument can be made for Dm7 and FM7. It may be the case that the behavior of these chords specifically is pulled out since they are all very common chords which means the rows in M corresponding to these chords have more granularity which would make patterns in how these chords are used more clear (higher signal to noise ratio).

The implementation computes the SVD for this graph (a 1010×1010 matrix) in ≈ 2.9 seconds. Going forward, we'll try to improve our multidimensional scaling technique to capture more of the behavior of the graph.

3.3 t-SNE

In the field of machine learning, multidimensional scaling techniques are often used to extract patterns as well as reduce the cost of computation for large data sets. SVD is ultimately bounded by its status as a linear transform and the chord graph—at least only allowing 2 dimensions—does not appear to be well described by a linear transform. Stochastic neighbor embedding is an iterative, non-linear technique for embedding data points in a high dimensional space H

into a lower dimensional space L. This is done by placing a point in L for each point in H and then shifting the points in L over multiple iterations to try and optimize some heuristic. T-distributed stochastic neighbor embedding is a version of SNE that tries to improve on other SNE algorithms by reducing cluttering and by having an easier to optimize heuristic [6]. In t-SNE, points are placed close together or far apart based on the pairwise affinity between points. The affinity between two points x_i and x_j is calculated as the conditional probability that x_i would pick x_j as its neighbor which is given by:

$$p_{j|i} = \frac{\exp\left(-||x_i - x_j||^2/2\sigma_i^2\right)}{\sum_{k \neq i} \exp\left(-||x_i - x_k||^2/2\sigma_i^2\right)} \tag{2}$$

where σ_i is the variance of a Gaussian distribution centered on point x_i[6]. The value of σ_i individually for each point based on the density of it's local neighbors and a global perplexity parameter passed at the start of the algorithm. A larger perplexity value generally corresponds to larger clusters being formed since each point will be more likely to consider nodes that are further away as neighbors. There is a lot of math involved that is better explained in the original paper here: [6]. As an additional step, since this calculation is essentially an n-body simulation using Euclidean distance between points, the algorithm can be made more efficient using the Barnes-Hut algorithm which uses quad trees to speed up n-body simulations.

In regards to our problem, we can potentially use t-SNE to place our chords in a 2D space and preserve the relative distances between chords that are close together in the 1010 dimensional chord space. While we could just apply t-SNE directly to our row-normalized adjacency matrix, one factor worth considering is the cost of calculating euclidean distances between many pairs of points in high dimensional space. To increase the running time of this step, we can try to reduce the dimensionality of the space to something more reasonable using a faster MDS method before running t-SNE. To do this, we can return to our SVD implementation from the previous section. However, instead of reducing our 1010 dimensional space directly down to a 2D space, we can reduce our space to a size where we can be confident that the behavior of the original space is still preserved and then run t-SNE on the lower dimensional set of data points. For this project, a reduction to a 35 dimensional space was chosen based on a sudden drop in the magnitude of the singular values past the 35th largest value. This can be seen in Figure 9. Another potential benefit of moving our data into a lower dimensional latent space first is that doing so may remove noise and small details that may prevent us from capturing the general overall structure of the space when running t-SNE. In the world of machine learning, the term "overfitting" would be used to describe the phenomenon we are trying to avoid.

Running the t-SNE algorithm with Barnes-Hut acceleration on our 35 dimensional chords takes ≈ 8.5 seconds and results in the graph shown in Figure 10. Finally, we are starting to get somewhere with this graph. There is a very clear shape beginning to materialize—it looks like we have a structure formed by strongly connected clusters where each cluster of nodes corresponds to similar

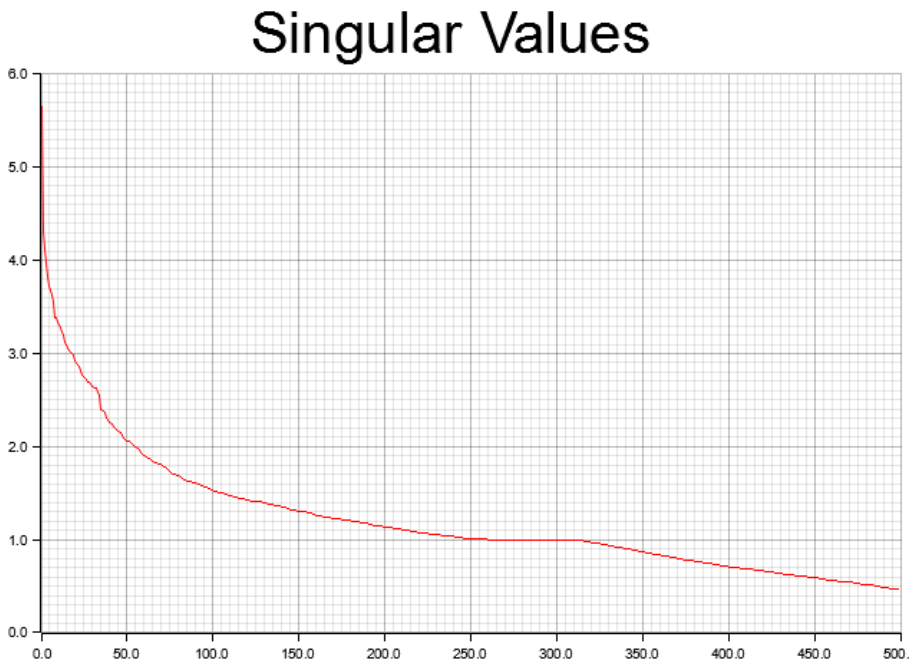

Figure 9: A graph showing the magnitudes of the first 500 largest singular values. We notice a sudden drop in magnitude after the 35th singular value.

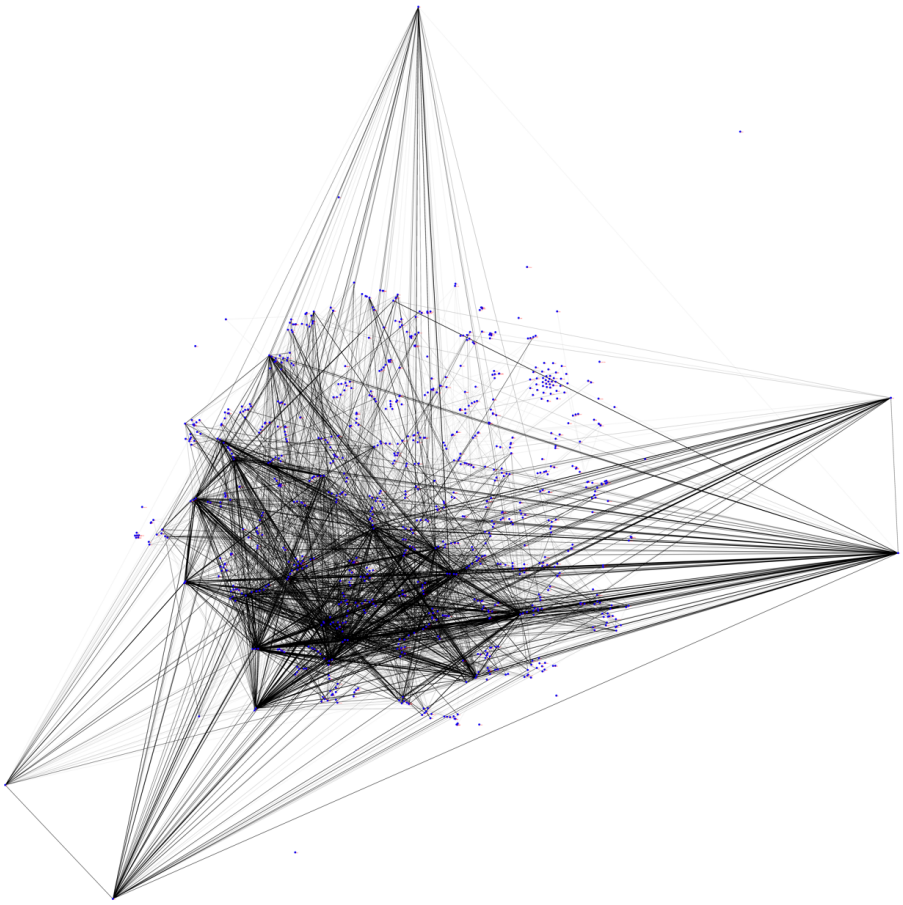

Figure 10: The graph drawn after using a combination of SVD and t-SNE to reduce the dimensionality of the row-normalized adjacency matrix (finally some progress!).

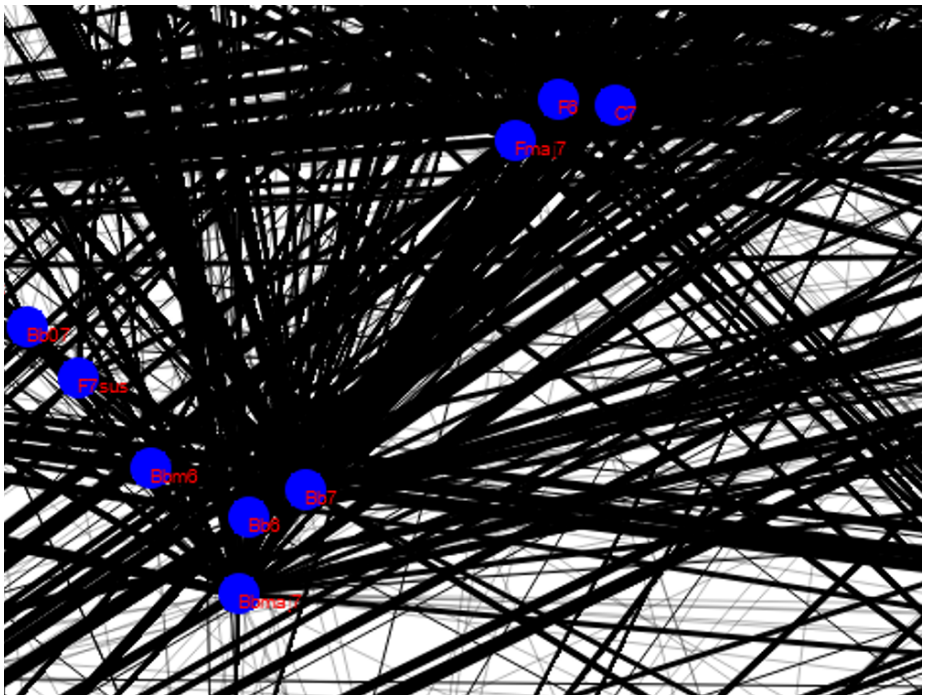

Figure 11: A zoomed in image of what the clusters look like.

chords. In Figure 11, we see that F6, Fmaj7, and C7 form one such cluster and Bb7, Bb6, Bbmaj7, Bbm6, F7sus, and Bb^{o}7 form another cluster. We're also starting to get our first real insights into the data. It's interesting that C7 is grouped with the F chords since C7 is the $\mathbf{V}^7$ of both of these chords. As stated in the music theory section of the introduction, the $\mathbf{V}^7$ chord has a very strong tendency to resolve to the I chord. This clustering pattern appears in other places as well (e.g. F7sus (a variant of the $\mathbf{V}^7$ for Bb) being grouped with the Bb chords) and it likely reflects this resolution tendency since many chord progressions that resolve to F chords will transition to C7 which will resolve to one of the F chords but may just jump directly to F instead. In addition, there are likely a lot of self-loop edges in the F chords since the chords are common and many songs will repeat chords multiple times in a row. The C7 chord constantly transitioning to F chords may look similar to these self-loop edges which may be another reason they are clustered together.

Overall, this drawing of the graph is a lot more structured and is a huge improvement over previous drawings.

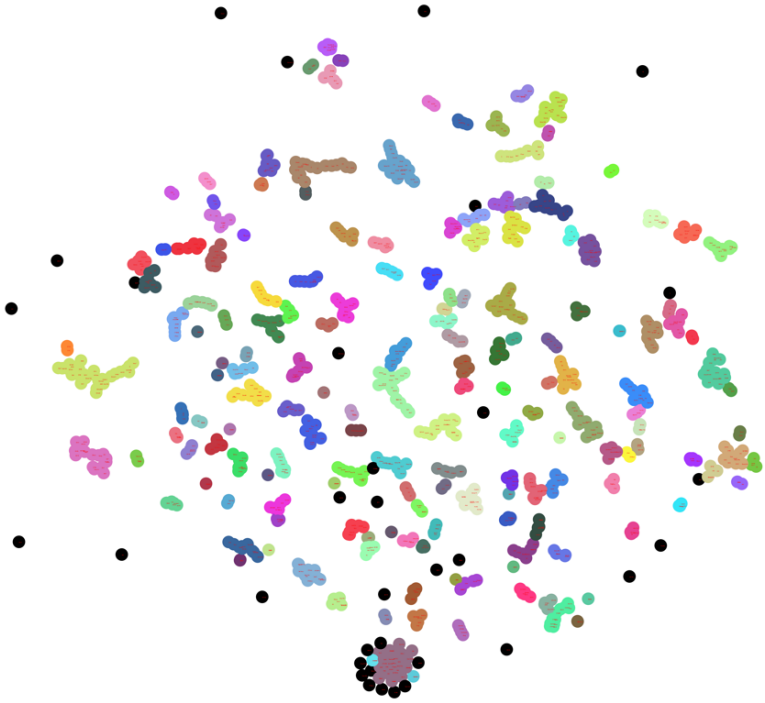

Figure 12: The result of running DBSCAN on the output of t-SNE. Random colors are chosen for clusters and outliers are colored black (a few outliers outside the image are omitted).

3.4 Clustering

The graph outputted from the t-SNE drawing appears to have a clustering pattern. We can identify the different clusters of similar chords in the graph and potentially reorganize the shape of our graph into something nicer while still maintaining cluster structures. Two simple clustering algorithms come to mind, k-means and DBSCAN. The DBSCAN algorithm appears more promising in this case since k-means is only well suited for convex clusters whereas DBSCAN is density-based and uses the distances between neighboring points to produce clusters. In our case, the clusters observed in the output of the t-SNE graph are not convex and often fall into curved strand-like shapes due to the nature of t-SNE (prioritizing placing neighbors near each other). DBSCAN requires the following parameters: the radius at which points are considered neighbors ε and the number of neighbor points required for a point to be considered a core point n [7]. Given these parameters, we can scan all points to find sets of core points and identify neighboring non-core points that are part of the same cluster as the core points. For our specific use case, we care about any group where two or

more chords are close enough to consider them as similar so we'll use $n = 2$. We can then try different values for ε until we're happy. After running DBSCAN on our data points, we get the clusters shown in Figure 12. This algorithm takes 2ms to run so it is quite fast.

3.5 Edge Bundling

[2]As we've seen in our previous graphs, although we can start to make out some of the structure of the graph, it is still very cluttered with edges. One of the more modern techniques for dealing with this is edge bundling where similar edges are bundled together to minimize the number of lines being drawn across the graph. For our algorithm, we'll be using a very basic iterative force-directed algorithm that treats the edges as springs and applies charge-like forces between segments along edges based on how similar they are (in terms of direction).

To do this, we'll first place n points indexed from 1 to n along each edge to divide the edge into $n + 1$ segments. At each iteration, for each point at index i along each edge, we apply attraction forces towards all other points of index i on all other edges. Let $e[i]$ mean the coordinates of the ith point along edge e. The attraction between two points at index i between two edges a and b is calculated using the following equation:

$$f_{i,(a,b)} = \frac{Q \cdot C}{||a[i] - b[i]||^2 + Q} \tag{3}$$

where Q is a charge parameter that determines the strength of the force and C is the compatibility between the two points. The compatibility between two points at index i is calculated using their neighboring points by taking the dot product of the average direction that the two edges are moving in at their respective i points. The compatibility is set to 0 if the dot product is negative. This way, if two edges are pointing in similar directions locally, the two points will be attracted to each other and if they are in opposite directions or are orthogonal, there will be no attraction force. The reason Q is in the denominator is to prevent the function from exploding when the distance between the two points gets close to 0. The force as a function of distance for $Q = 10,000$ and $C = 1$ can be seen in Figure 13. This equation is quite messy and could definitely be improved upon with further experience. After calculating the attraction force, we must also calculate the tension force of each individual edge. The tension can be calculated using a linear function of the length of each segment. For each point along an edge, we calculate the distance from that point to it's neighbors and add forces to the point to push it away or pull it towards it's neighbors depending on the difference between the expected distance (the original length of segments before running iterations) and the actual distance. A spring constant is used to determine the strength of the tensile forces. From here, we just add up all the forces for each point and move them by some step size (which can decrease over time) and repeat the process until we're happy.

[2]This phase of the project is very experimental since I didn't have much knowledge (and time) about edge bundling at the time and just wanted to try my own thing

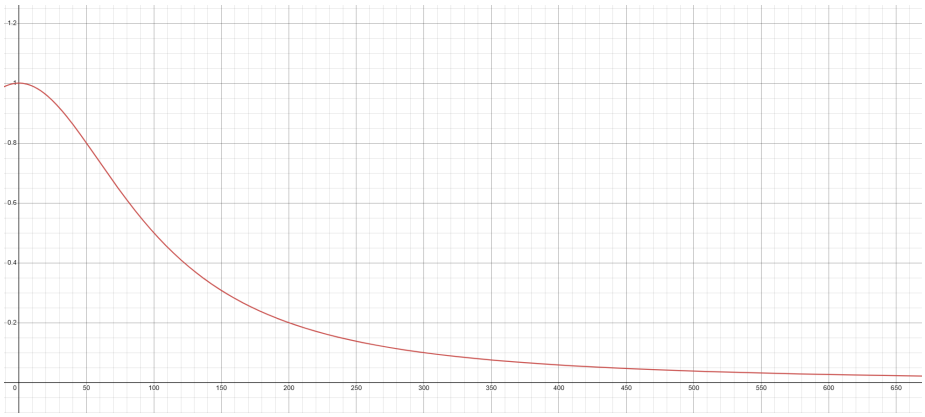

Figure 13: A graph of the strength of the attraction force between two points along two edges with $Q = 10,000$ and $C = 1$

For our graph in particular, to prevent edges from overlapping nodes, we can use our identified clusters from the previous section to reconstruct our circular graph while preserving groups of similar chords along the circle. If we now run our edge bundling algorithm, we get a graph that looks like Figure 14. Unfortunately, our bundled edges ended up very tangled not smooth looking.

At this point however, as there was not much time left, to attempt to produce an edge bundled graph that looked decent, the graph was initialized by dragging the edges towards the center before running the edge bundling algorithm. The resulting graph was then smoothed and is shown in Figure 15.

This method of edge bundling ended up being pretty terrible in terms of results and running time but it was a good learning experience nonetheless. If this were to be done again with more time and experience, no doubt it could be made to look a lot nicer.

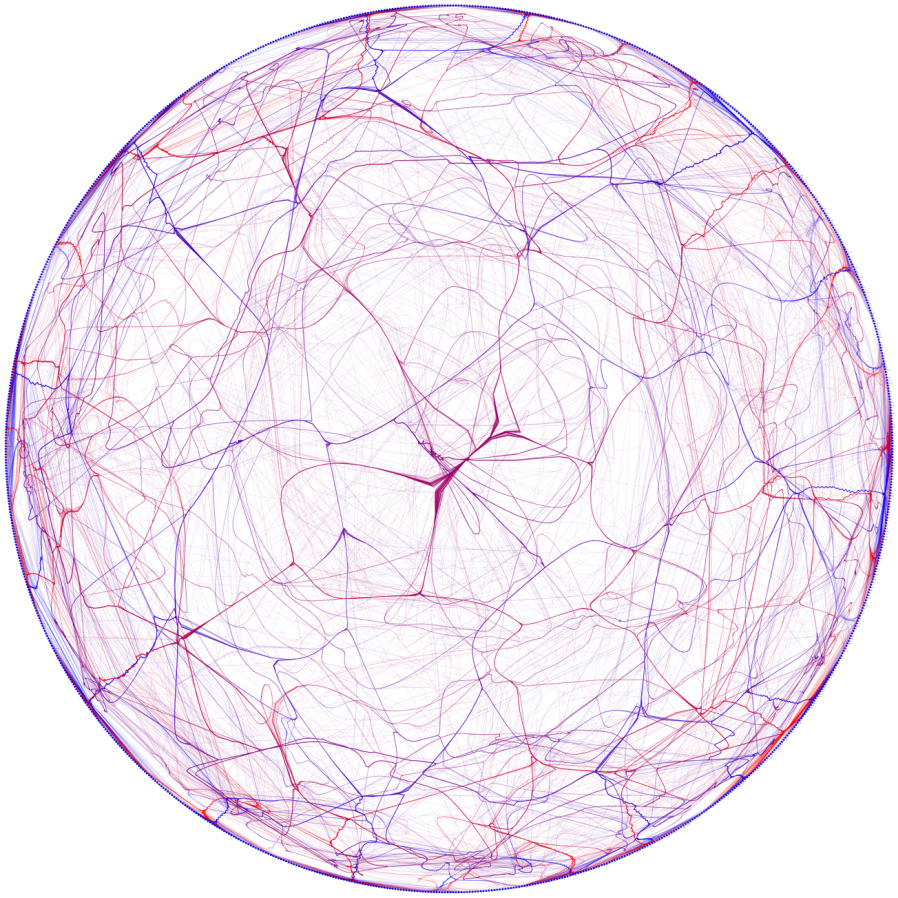

Figure 14: A graph drawn using our edge bundling algorithm using 1000 iterations. Directional edges are drawn from blue to red (outgoing = blue, incoming = red). This graph is pretty terrible from an info-vis standpoint since the bundled edges are too chaotic and hard to follow but it is a somewhat cool effect.

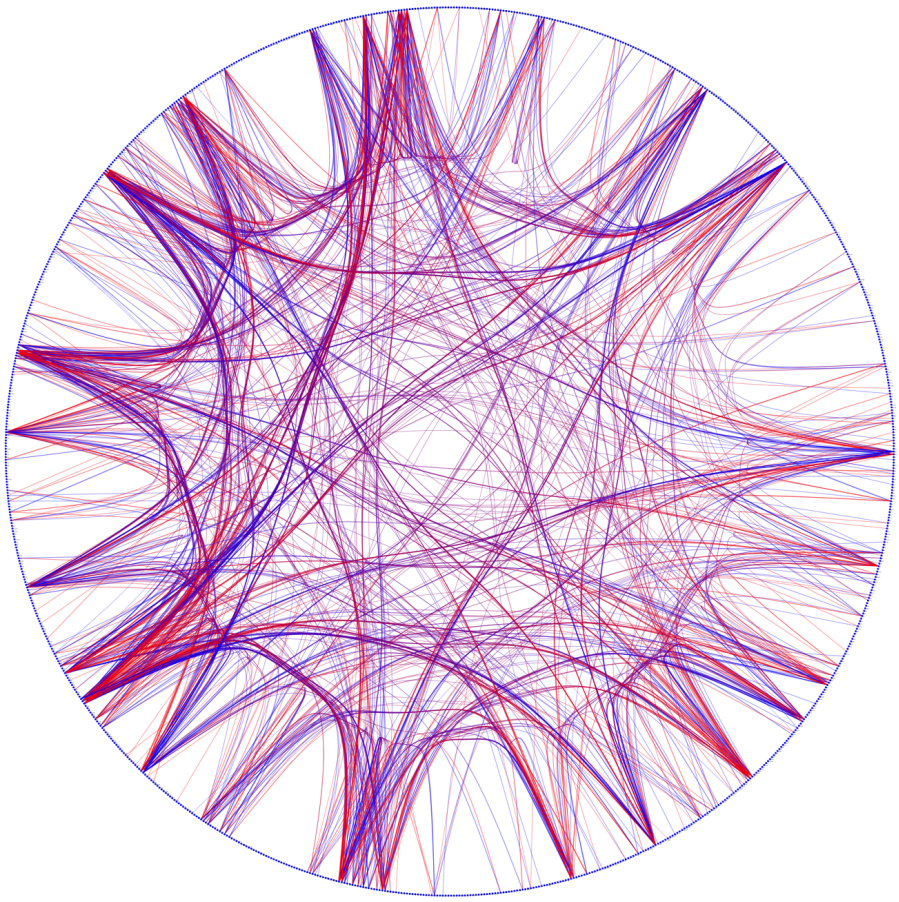

Figure 15: A graph drawn using the edge bundling algorithm with edges initialized to create a more aesthetic image. After running the algorithm, the edges were smoothed to remove unnecessary kinks and bends. There is some apparent structure in this graph but it's still quite chaotic.

4 Conclusion

In the end, there wasn't really enough time to really get into the information visualization aspect of drawing this graph. In addition, jazz is and always will be quite chaotic in terms of chord progressions. The nature of jazz is improvisation and "mistakes" are what add the jazzy colour to an otherwise straight forward set of chords. "Any attempt to arrive at a precise, all encompassing definition of jazz is probably futile" [5]. In all the graphs, the labels of the nodes are quite small since the graph is so large. Perhaps, a better way to visualize this data would be something interactive where individual clusters could be zoomed in on

to see which chords form that cluster. In addition, more common chords could be made larger to reflect their prominence and the clustering could potentially relate less common chords with their more common counterparts. Another approach could be to start with the circle of fifths from Figure 1 as an initial structure for the graph and then build the rest of the chords around them. One observation made earlier is that clusters often included their $\mathbf{V}^7$ chords. These chords are obviously different and should probably not be in the same group. This could potentially be solved by removing edges that loop back to the same node or by using the previous n transitions for each chord instead of just 1 transition (although with larger n, you would potentially need a larger data set to better satisfy the coupon collector problem). It would be an interesting idea to try drawing a graph for something more structured like classical music to see if the techniques work well for that before continuing with jazz chords.

References

[1] S. Kostka and D. Payne, *Tonal Harmony, with an Introduction to Twentieth-century Music*. McGraw-Hill, 2000.

[2] mikeoliphant, "Jazzstandards." https://github.com/mikeoliphant/JazzStandards, 2023.

[3] J. Bruss, "chord_visualization." https://github.com/Amarantheum/chord_visualization, 2023.

[4] P. Barrow, "The importance of music theory." https://resoundschool.com/importance-music-theory/, Mar 2017.

[5] G. Schuller, "Jazz." https://www.britannica.com/art/jazz, Nov 2023.

[6] L. van der Maaten and G. Hinton, "Visualizing data using t-sne," *Journal of Machine Learning Research 9*, 2008.

[7] M. Ester *et al.*, "A density-based algorithm for discorvering clusters," *KDD-96: Proceedings*, 1996.